Workshop Mobile Applications Framework

GESTURE LAB

ANALISIS IMPLEMENTASI GESTURE PADA APLIKASI FLUTTER (GESTURE LAB)

Muhammad Shobri
E41240316
Golongan A

gesture_apps

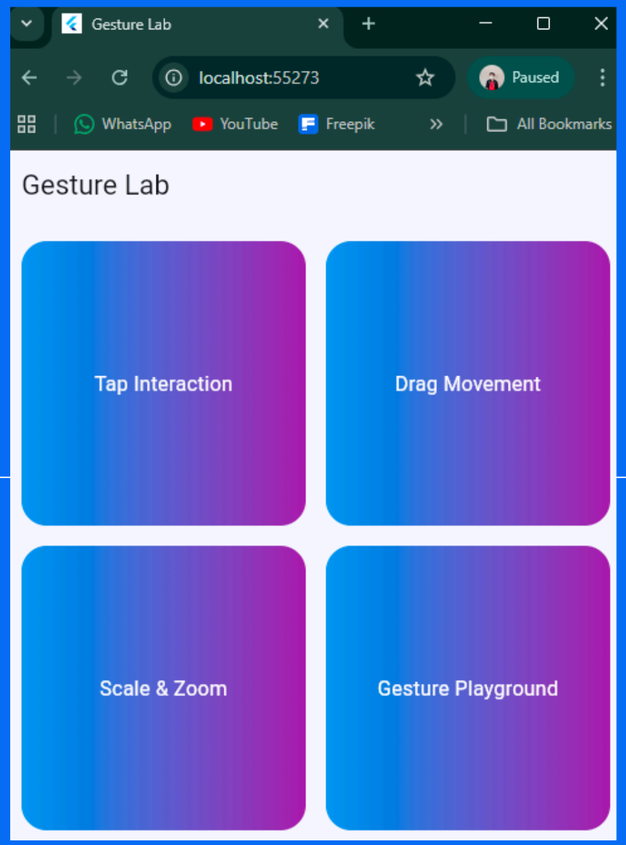
widget_test.dart | home_page.dart | main.dart

gesture > lib > main.dart > ...

```dart
import 'package:flutter/material.dart';
import 'home_page.dart';

Run | Debug | Profile
void main() {
  runApp(GestureApp());
}

class GestureApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gesture Lab',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.blue),
      home: HomePage(),
    ); // MaterialApp
  }
}
```



MAIN.DART

halamn utama

File ini adalah titik awal aplikasi. runApp() menjalankan GestureApp, lalu MaterialApp mengatur tema, judul, dan halaman awal (HomePage). ini menunjukkan pembagian yang jjelas atara konfigurasi aplikasi dan tampilan utama

widget_test.dart | home_page.dart | main.dart

gesture > lib > home_page.dart > HomePage > build

```dart
import 'package:flutter/material.dart';
import 'pages/tap_page.dart';
import 'pages/drag_page.dart';
import 'pages/scale_page.dart';
import 'pages/playground_page.dart';

class HomePage extends StatelessWidget {
  final menus = [
    {"title": "Tap Interaction", "page": TapPage()},
    {"title": "Drag Movement", "page": DragPage()},
    {"title": "Scale & Zoom", "page": ScalePage()},
    {"title": "Gesture Playground", "page": PlaygroundPage()},
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Gesture Lab")),
      body: Padding(
        padding: EdgeInsets.all(16),
        child: GridView.builder(
          itemCount: menus.length,
          gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 2,
            crossAxisSpacing: 16,
            mainAxisSpacing: 16,
          ), // SliverGridDelegateWithFixedCrossAxisCount
          itemBuilder: (_, i) {
            return GestureDetector(
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (_) => menus[i]["page"] as Widget),
                );
              },
```

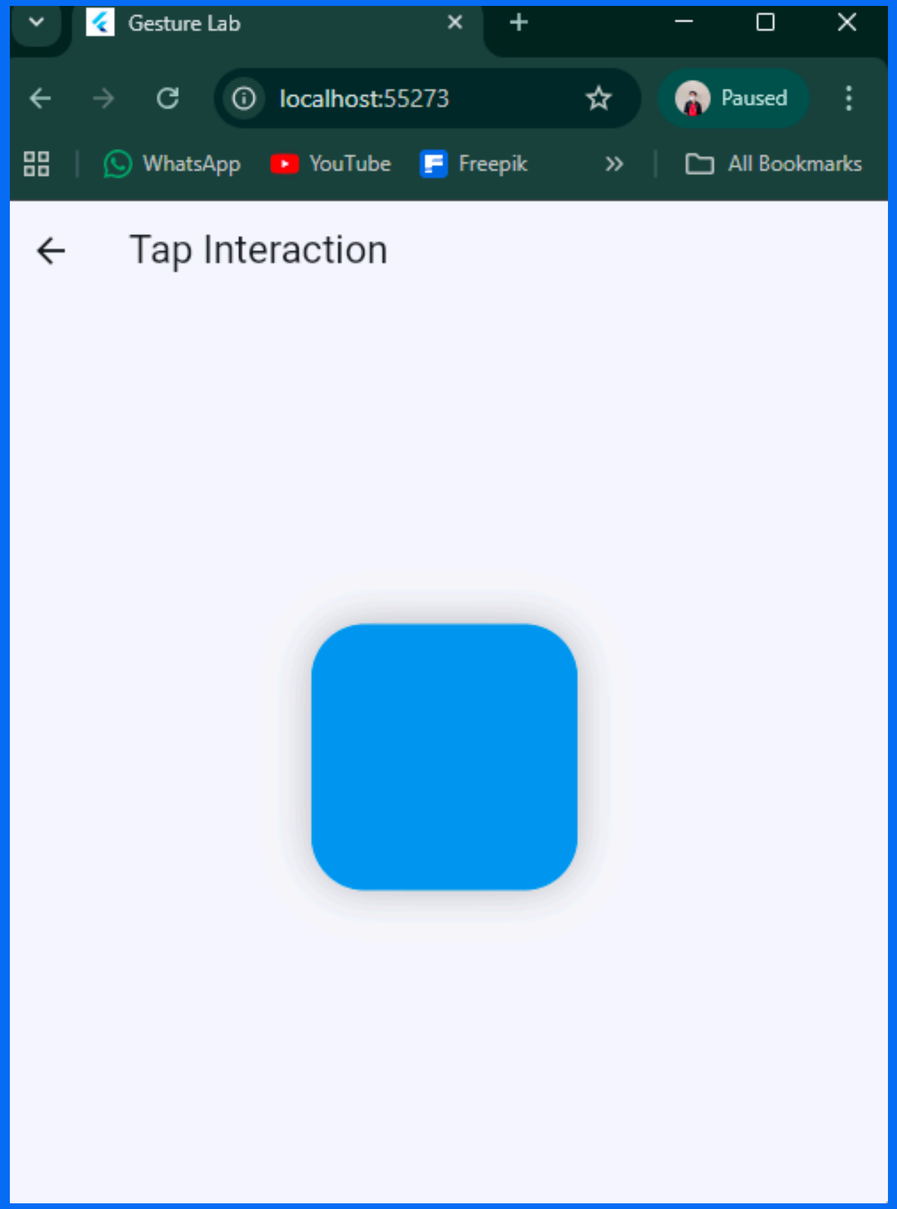
```dart
            child: AnimatedContainer(
              duration: Duration(milliseconds: 300),
              decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(20),
                gradient: LinearGradient(
                  colors: [Colors.blue, Colors.purple],
                ), // LinearGradient
              ), // BoxDecoration
              child: Center(
                child: Text(
                  menus[i]["title"] as String,
                  style: TextStyle(color: Colors.white, fontSize: 16),
                  textAlign: TextAlign.center,
                ), // Text
              ), // Center
            ), // AnimatedContainer
          ); // GestureDetector
        },
      ), // GridView.builder
    ), // Padding
  ); // Scaffold
}
}
```

Rimberio

kode ini berfungsi sebagai halaman utama yang menampilkan berbagai menu fitur gesture seperti Tap, Drag, Scale, dan Playground. Data menu disimpan dalam list menus, sehingga lebih fleksibel dan memudahkan pengembangan. Dengan cara ini, penambahan fitur baru dapat dilakukan tanpa mengubah struktur utama aplikasi.

Tampilan menu dibuat menggunakan GridView.builder dengan dua kolom, yang menurut saya cukup efektif karena dapat menampilkan item secara dinamis sesuai jumlah data. Selain itu, penggunaan grid membuat tampilan lebih rapi dan mudah dipahami oleh pengguna.

Di sisi interaksi, setiap menu dibungkus dengan GestureDetector untuk mendeteksi sentuhan pengguna. Ketika menu ditekan, event onTap akan dijalankan dan mengarahkan ke halaman lain menggunakan Navigator.push(). Menurut saya, hal ini menunjukkan bahwa aplikasi sudah menerapkan konsep event-driven dengan baik, ditambah penggunaan AnimatedContainer dan LinearGradient yang membuat tampilan lebih menarik dan responsif.



Tap Interaction

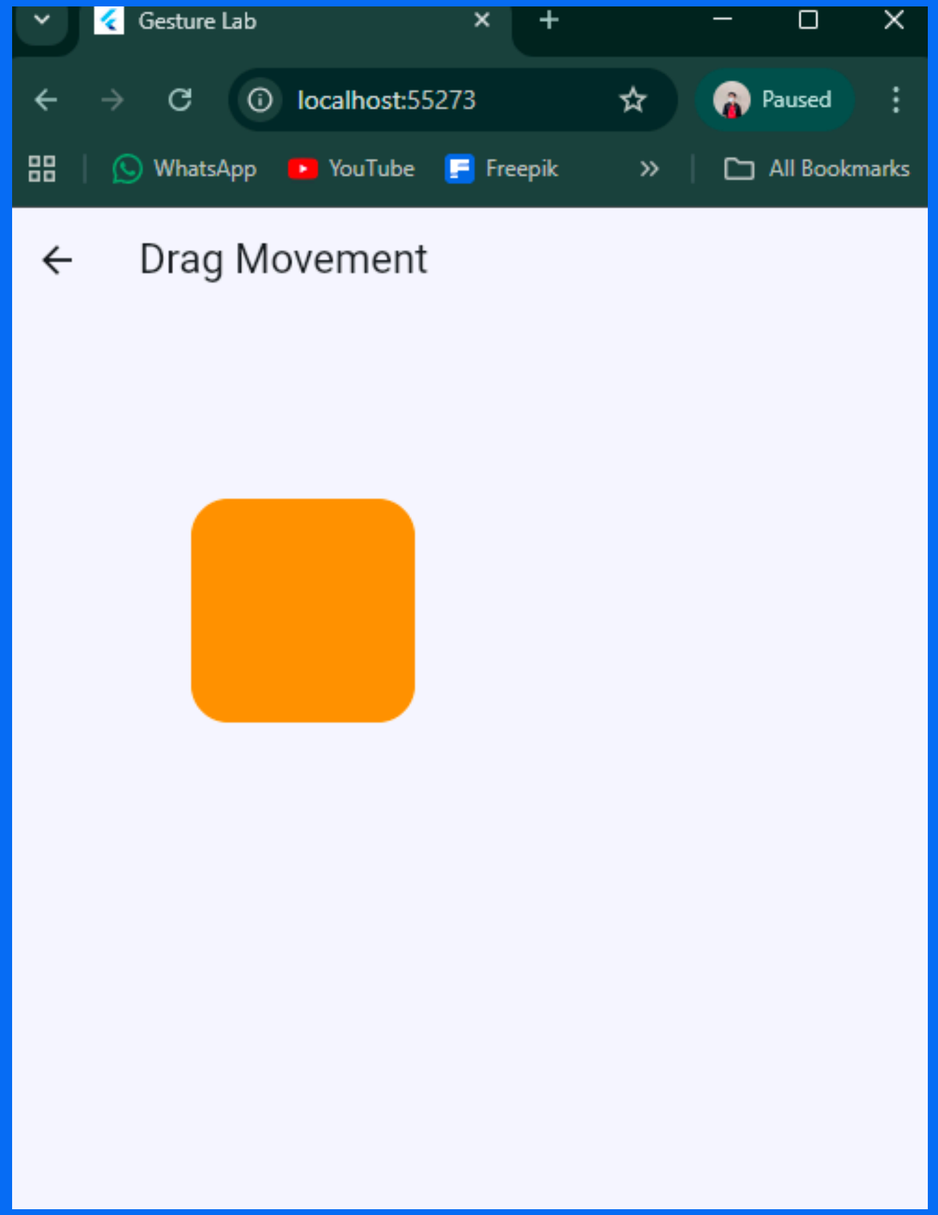
Tap Interaction menggambarkan bagaimana sebuah elemen pada layar merespon sentuhan dari pengguna. Pada kondisi awal, objek ditampilkan dalam ukuran kecil dengan warna tertentu. Ketika pengguna melakukan tap, sistem langsung memberikan respon berupa perubahan ukuran dan warna sebagai bentuk umpan balik visual.

Respon ini terjadi karena adanya GestureDetector yang menangkap aksi tap melalui onTap. Saat interaksi terdeteksi, fungsi setState() dijalankan untuk memperbarui nilai properti seperti ukuran dan warna. Perubahan tersebut kemudian ditampilkan kembali ke layar sehingga pengguna dapat melihat perbedaannya secara langsung.

Agar perubahan terlihat lebih halus dan tidak kaku, digunakan AnimatedContainer yang memberikan efek transisi. Dengan begitu, perubahan dari kondisi awal ke kondisi akhir terjadi secara bertahap dan lebih nyaman dilihat.

Alur Singkat:

- Awal: objek kecil (warna awal)
- Tap → gesture terdeteksi
- onTap dijalankan
- setState() mengubah ukuran & warna
- AnimatedContainer menampilkan animasi
- Hasil: objek berubah (lebih besar & warna berbeda)



Drag movement

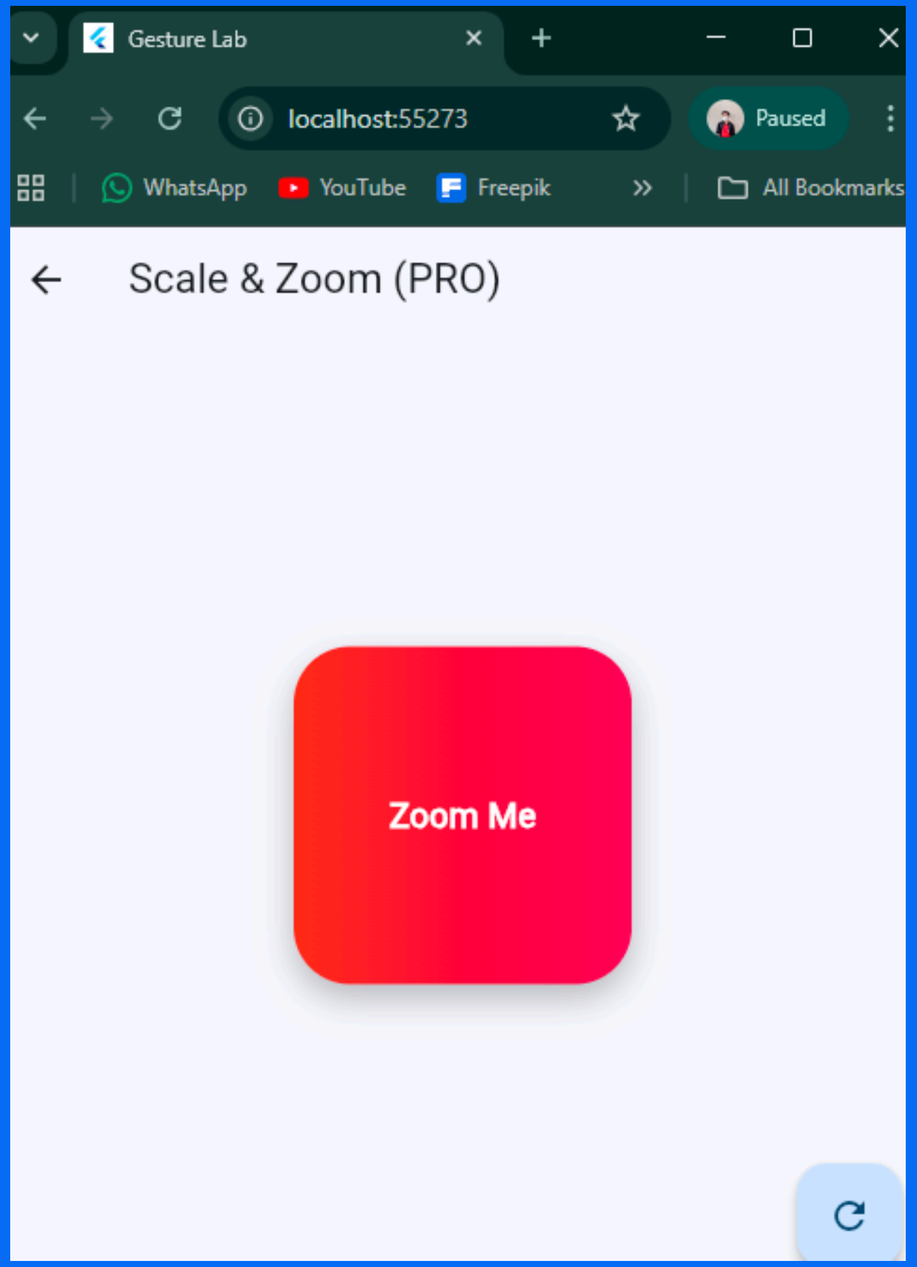
Drag Movement memperlihatkan bagaimana sebuah objek dapat dipindahkan dengan mengikuti gerakan jari pengguna di layar. Pada awalnya, objek berada pada posisi tetap. Ketika pengguna menyentuh dan mulai menggeser, objek akan bergerak sesuai arah yang diberikan.

Pergerakan ini terjadi karena sistem mendeteksi gesture drag seperti onPanUpdate. Setiap perubahan posisi jari akan langsung diproses dan disimpan menggunakan setState(), sehingga posisi objek terus diperbarui secara real-time.

Dengan mekanisme ini, objek terasa “menempel” pada jari pengguna, sehingga interaksi menjadi lebih intuitif dan responsif.

Alur Singkat:

- Awal: objek di posisi tetap
- Drag → pergerakan terdeteksi
- onPanUpdate dijalankan
- setState() memperbarui posisi
- Objek mengikuti gerakan
- Hasil: objek berpindah
-



Scale & Zoom

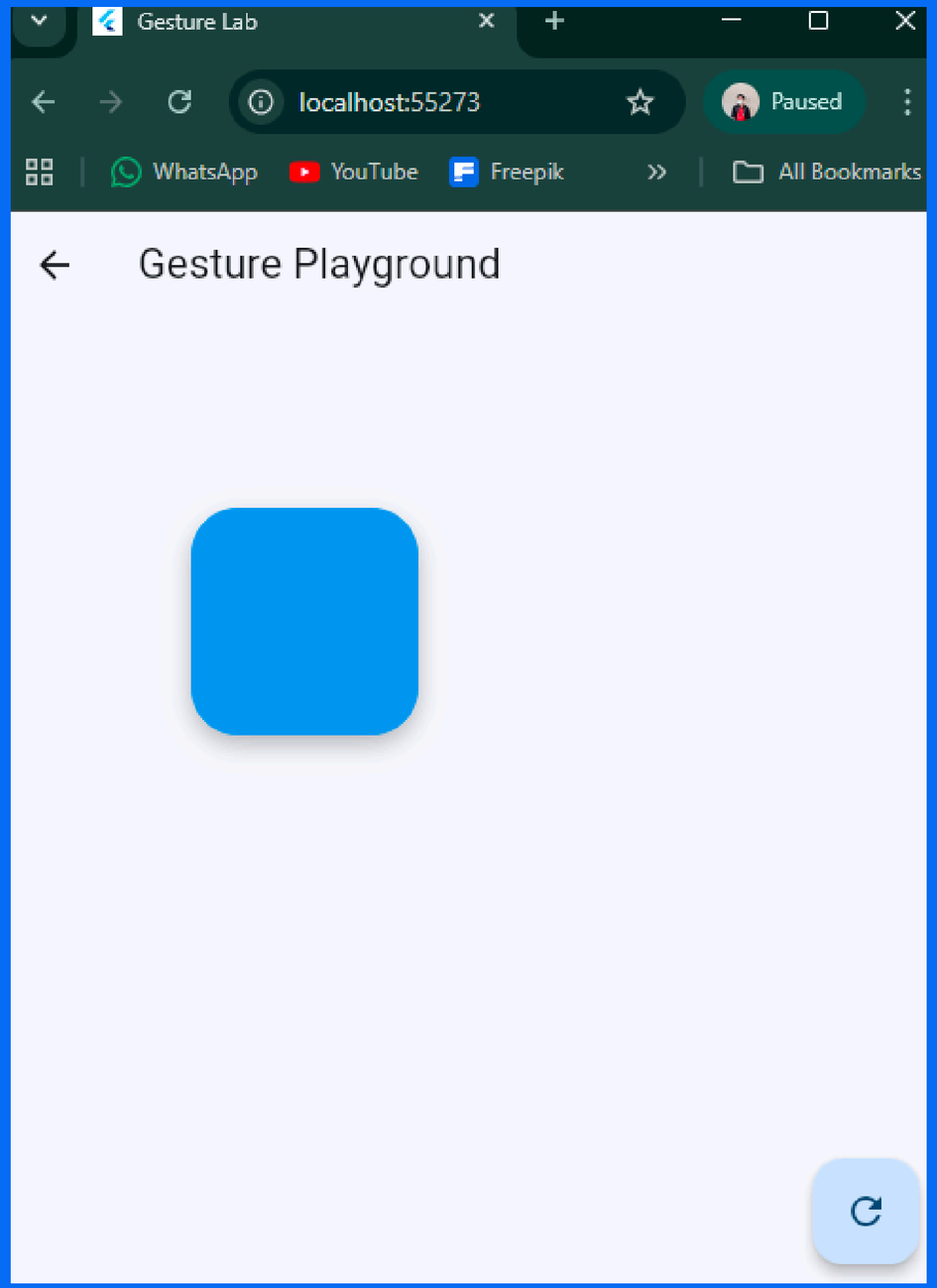
Scale & Zoom digunakan untuk mengatur ukuran objek melalui gesture dua jari. Awalnya, objek ditampilkan dalam ukuran normal. Ketika pengguna melakukan gerakan memperbesar atau memperkecil (pinch), ukuran objek akan berubah mengikuti gerakan tersebut.

Gesture ini dideteksi menggunakan event seperti onScaleUpdate, yang membaca perubahan jarak antar jari pengguna. Nilai skala kemudian diperbarui menggunakan setState(), sehingga ukuran objek berubah secara langsung.

Interaksi ini memberikan pengalaman yang lebih fleksibel, terutama ketika pengguna ingin melihat detail atau menyesuaikan tampilan sesuai kebutuhan.

Alur Singkat:

- Awal: ukuran normal
- Pinch → gesture terdeteksi
- onScaleUpdate dijalankan
- setState() mengubah skala
- Ukuran objek menyesuaikan
- Hasil: objek zoom in / zoom out



Gesture Playground

Gesture Playground merupakan area eksplorasi yang menggabungkan berbagai jenis gesture dalam satu halaman. Di sini, pengguna tidak hanya melakukan satu jenis interaksi, tetapi bisa mencoba beberapa gesture sekaligus dalam satu waktu.
Setiap gesture yang dilakukan akan diproses sesuai jenisnya, baik itu tap, drag, maupun scale. Sistem akan merespon setiap interaksi secara langsung, sehingga pengguna dapat melihat perbedaan cara kerja masing-masing gesture.
Fitur ini sangat membantu dalam memahami bagaimana berbagai gesture dapat bekerja secara bersamaan dalam satu aplikasi.
Alur Singkat:

- Awal: tampilan utama playground
- User mencoba berbagai gesture
- Sistem mendeteksi tiap interaksi
- setState() memperbarui kondisi
- Respon ditampilkan secara langsung
- Hasil: kombinasi gesture dalam satu halaman